

Связи между таблицами: 1:1, 1:N, M:N

Связываем сущности Internet Shop через PRIMARY KEY и FOREIGN KEY

Результаты занятия



Объяснять понятие кардинальности связи



Различать связи 1:1, 1:N и M:N



Определять, в какой таблице должен находиться FOREIGN KEY



Реализовывать связь один к одному



Реализовывать связь один ко многим



Создавать промежуточную таблицу для связи многие ко многим



Поддерживать ссылочную целостность средствами PostgreSQL



Проектировать связи между сущностями Internet Shop

Учебные вопросы

- 1 Кардинальность и внешние ключи** Что означает связь между таблицами и как её описывают
- 2 Связь 1:1** Одна строка одной таблицы связана максимум с одной строкой другой
- 3 Связь 1:N** Одна родительская строка может быть связана со многими дочерними
- 4 Связь M:N** Многие строки одной таблицы связаны со многими строками другой
- 5 Связи в Internet Shop** users, orders, products, categories и order_items в единой схеме



Учебный вопрос 1. Кардинальность и внешние ключи

Связь показывает, сколько строк одной таблицы может соответствовать строкам другой

Зачем связывать таблицы

- Разные сущности хранятся в отдельных таблицах.
- Но объекты предметной области взаимодействуют друг с другом.
- Товар относится к категории.
- Пользователь создаёт заказы.
- Заказ содержит товары.
- Связи позволяют представить эти отношения в реляционной базе данных.

Что такое кардинальность

- Кардинальность показывает, сколько экземпляров одной сущности может быть связано с экземплярами другой сущности.
- 1:1 — один объект связан максимум с одним объектом.
- 1:N — один объект может быть связан со многими объектами.
- M:N — многие объекты могут быть связаны со многими объектами.
- Кардинальность определяется правилами предметной области.
- Она влияет на структуру таблиц и расположение внешних ключей.

Три основных типа связей

Связь	Смысл	Пример Internet Shop
1:1	Одна строка ↔ одна строка	User ↔ User Profile
1:N	Одна строка → много строк	Category → Products
M:N	Много строк ↔ много строк	Orders ↔ Products

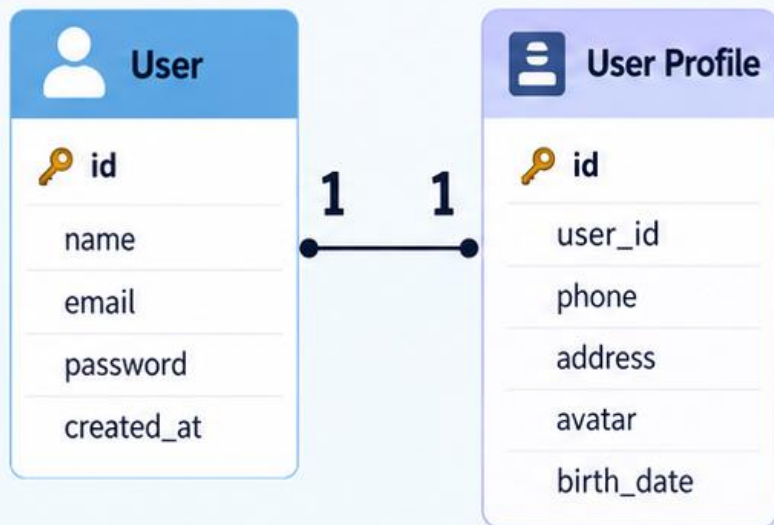


1:1, 1:N и M:N на примере Internet Shop

Разные типы связей между сущностями в интернет-магазине

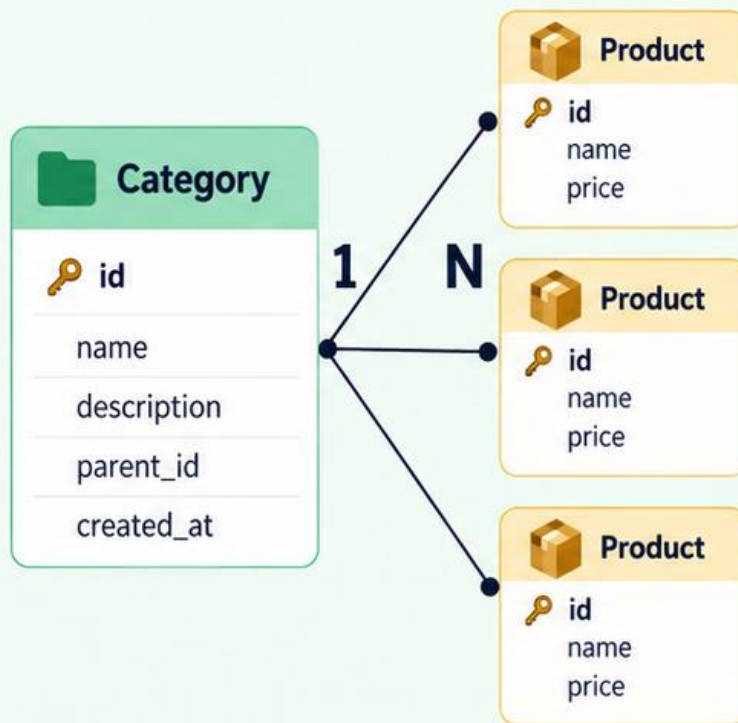
Понятные
базы данных
для реальных задач

Связь 1:1



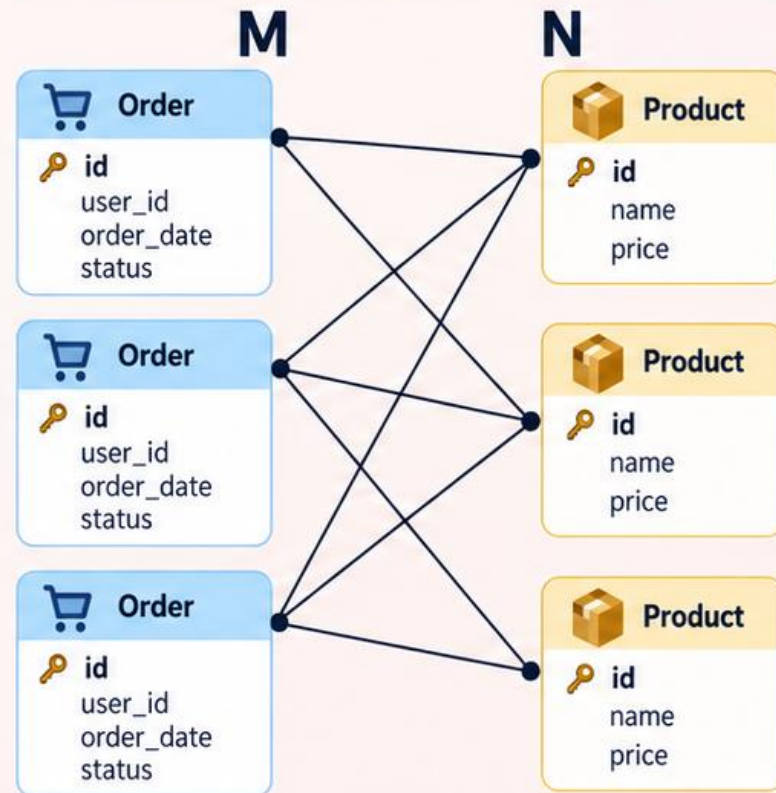
Один пользователь —
максимум один профиль

Связь 1:N



Одна категория —
несколько товаров

Связь M:N



Несколько заказов ↔
несколько товаров

PRIMARY KEY и FOREIGN KEY

- PRIMARY KEY однозначно идентифицирует строку таблицы.
- FOREIGN KEY хранит ссылку на ключ другой таблицы.
- Внешний ключ связывает строки разных таблиц.
- PostgreSQL проверяет существование связанной строки.
- Связь определяется не названием столбца, а ограничением FOREIGN KEY.

Простейшая связь через FOREIGN KEY

SQL

```
CREATE TABLE products (  
  id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  name VARCHAR(150) NOT NULL,  
  category_id INTEGER NOT NULL REFERENCES categories(  
);
```

Что важно

- products.id идентифицирует товар.
- products.category_id хранит ссылку на категорию.
- REFERENCES categories(id) создаёт внешний ключ.
- PostgreSQL не разрешит category_id, которого нет в categories.

Родительская и дочерняя таблица

- Таблицу, на ключ которой ссылаются, часто называют родительской.
- Таблицу с FOREIGN KEY называют дочерней.
- В связи Category → Product родительская таблица — categories.
- Дочерняя таблица — products.
- Дочерняя строка хранит идентификатор родительской строки.



Учебный вопрос 2. Связь 1:1

Одной строке основной таблицы соответствует максимум одна строка дополнительной

Что означает 1:1

- Связь 1:1 означает, что одной строке таблицы A может соответствовать не более одной строки таблицы B, и наоборот.
- Связь удобна для разделения данных одной сущности на несколько таблиц.
- Основная таблица содержит базовые данные.
- Дополнительная таблица содержит расширенные или необязательные данные.
- Внешний ключ должен быть уникальным.
- Один из способов — сделать FOREIGN KEY одновременно PRIMARY KEY.

Пример User и User Profile

● users

Основные данные: id, name, email.

● user_profiles

Дополнительные данные: user_id, phone, address.

● Правило

У одного пользователя не может быть двух профилей.

Таблица users

SQL

```
CREATE TABLE users (  
  id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(150) NOT NULL UNIQUE  
);
```

Что важно

- Каждый пользователь имеет уникальный id.
- email обязателен.
- UNIQUE не позволяет зарегистрировать одинаковый email дважды.

Связь 1:1 через PRIMARY KEY + FOREIGN KEY

SQL

```
CREATE TABLE user_profiles (  
    user_id INTEGER PRIMARY KEY REFERENCES users(id),  
    phone VARCHAR(30),  
    address VARCHAR(250)  
);
```

Что важно

- user_id является PRIMARY KEY таблицы user_profiles.
- Поэтому один user_id не может повторяться.
- Одновременно user_id является FOREIGN KEY на users(id).
- Таким образом одному User соответствует максимум один User Profile.

Почему второй профиль создать нельзя

Запрос: Повторно создать user_profile для user_id = 1

SQL

```
INSERT INTO user_profiles  
  (user_id, phone)  
VALUES  
  (1, '+7 700 111 22 33');
```

Результат

ERROR: duplicate key value
violates unique constraint

Пояснение

- PRIMARY KEY user_id гарантирует, что для одного пользователя существует не более одной строки профиля.

Связь 1:1: User ↔ User Profile

Каждый пользователь имеет ровно один профиль, и каждый профиль принадлежит ровно одному пользователю.



users

Пользователи системы

id PK	name	email
1	Иван Петров	ivan@example.com
2	Мария Сидорова	maria@example.com
3	Алексей Кузнецов	alex@example.com
...	...	...

1

1

user_profiles.user_id → users.id

Связь 1:1

Профиль ссылается на пользователя по user_id, который является первичным ключом в users.



user_profiles

Профили пользователей

user_id PK FK	phone	address
1	+7 900 123-45-67	г. Москва, ул. Ленина, д. 10
1	+7 900 999-99-99	г. Москва, ул. Пушкина, д. 5
2	+7 911 111-22-33	г. Санкт-Петербург, Невский пр., д. 1
...	...	...

✗ нельзя: второй профиль для user_id = 1



Один пользователь — максимум один профиль.

Для каждого user.id может существовать только одна запись в user_profiles с таким же user_id.

Когда использовать 1:1

- Когда часть данных является необязательной.
- Когда дополнительные данные удобно отделить от основной таблицы.
- Когда различные части данных имеют разные правила доступа.
- Когда одна большая таблица логически содержит несколько разных блоков данных.
- Не следует автоматически разделять каждую сущность на несколько таблиц без практической причины.

1:1 не обязательно означает наличие двух строк

- Пользователь может существовать без строки `user_profiles`.
- Но если профиль создан, он относится только к одному пользователю.
- И у одного пользователя не может быть второго профиля.
- Поэтому на практике связь часто означает $1 \rightarrow 0..1$.
- Кардинальность описывает допустимое количество связанных объектов.



Учебный вопрос 3. Связь 1:N

Одна родительская строка может иметь множество дочерних строк

Что означает 1:N

- Связь 1:N означает, что одна строка родительской таблицы может быть связана со многими строками дочерней таблицы.
- Каждая дочерняя строка обычно относится к одному родителю.
- FOREIGN KEY размещается на стороне N.
- Одна Category может содержать много Products.
- Один User может создать много Orders.
- Это один из наиболее распространённых типов связей.

Примеры 1:N

Сторона 1	Сторона N	FOREIGN KEY
categories	products	products.category_id
users	orders	orders.user_id

Главное правило связи 1:N

- Внешний ключ размещается в таблице на стороне N.
- products хранит category_id, потому что товаров в категории много.
- orders хранит user_id, потому что заказов пользователя может быть много.
- В родительской таблице не нужен массив идентификаторов дочерних строк.
- Связь определяется значением FOREIGN KEY каждой дочерней строки.

Главная идея

1:N → FOREIGN KEY находится на стороне N.

Category → Products

SQL

```
CREATE TABLE products (  
  id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  name VARCHAR(150) NOT NULL,  
  price INTEGER NOT NULL CHECK (price > 0),  
  category_id INTEGER NOT NULL REFERENCES categories(  
  in_stock BOOLEAN NOT NULL DEFAULT true  
);
```

Что важно

- Одна категория может использоваться во многих строках products.
- Один товар содержит только один category_id.
- Много товаров могут иметь одинаковый category_id.
- Это и формирует связь 1:N.

User → Orders

SQL

```
CREATE TABLE orders (  
  id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  user_id INTEGER NOT NULL REFERENCES users(id),  
  status VARCHAR(30) NOT NULL DEFAULT 'new',  
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

Что важно

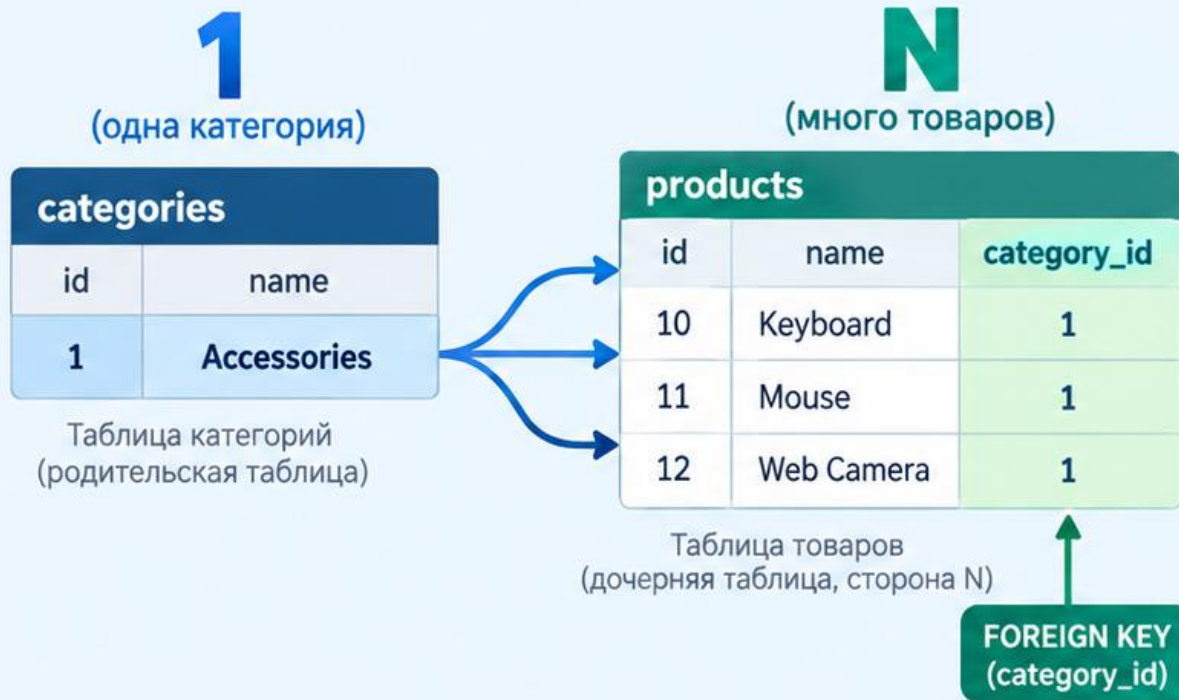
- Каждый order содержит user_id.
- Один user_id может встречаться во многих заказах.
- Каждый заказ принадлежит одному существующему пользователю.
- PostgreSQL проверяет ссылочную целостность.

Связь 1:N: Category → Products и User → Orders

Один объект на стороне 1 может быть связан со многими объектами на стороне N.

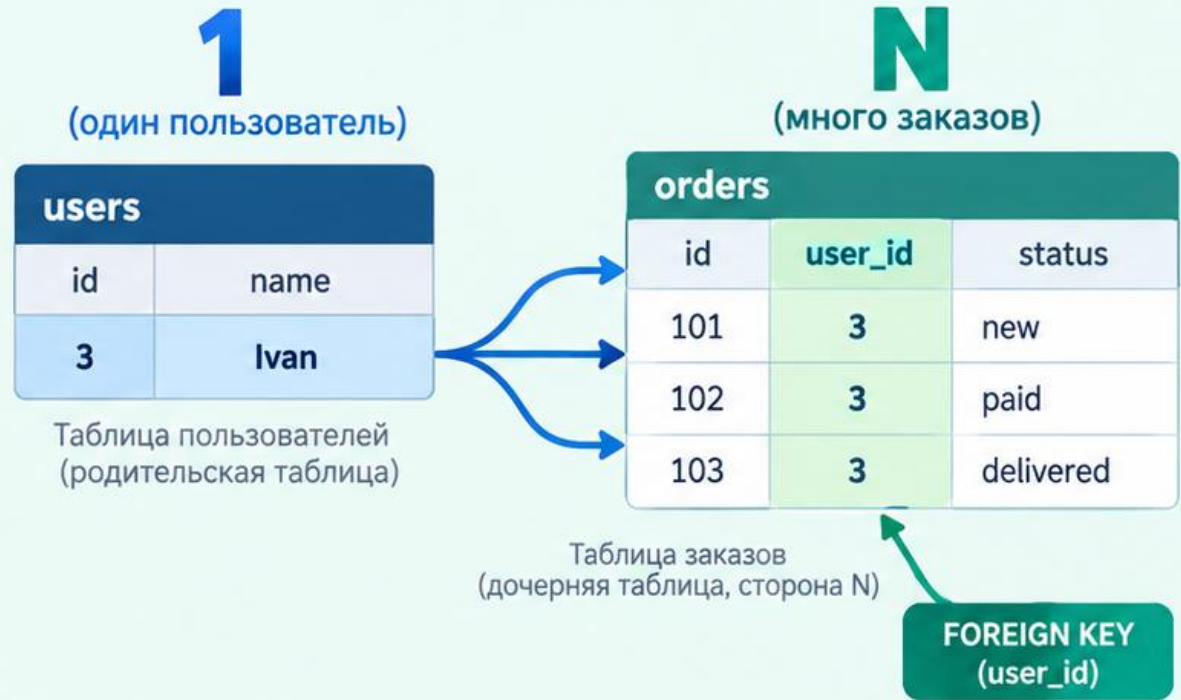
Пример 1: Одна категория — много товаров

Одна категория может содержать много товаров.



Пример 2: Один пользователь — много заказов

Один пользователь может оформить много заказов.



FOREIGN KEY находится на стороне N

В связи 1:N внешний ключ хранится в таблице на стороне многих (N).

Один пользователь и несколько заказов

Запрос: Получить заказы пользователя с id = 3

SQL

```
SELECT
  id,
  user_id,
  status,
  created_at
FROM orders
WHERE user_id = 3
ORDER BY created_at DESC;
```

Результат

12		3		new		...
8		3		paid		...
2		3		delivered		...

Пояснение

- Несколько строк orders содержат одинаковый user_id = 3 и относятся к одному пользователю.

1:N и JOIN

SQL

```
SELECT p.name, c.name AS category
FROM products AS p
JOIN categories AS c
    ON c.id = p.category_id;
```

Что важно

- FOREIGN KEY хранит связь между строками.
- JOIN позволяет получить данные связанных таблиц в одном результате.
- Например, можно получить товар и название его категории.
- Или заказ и имя пользователя.
- Связь существует независимо от того, выполняем мы JOIN или нет.



Учебный вопрос 4. Связь M:N

Многие строки одной таблицы связаны со многими строками другой

Что означает M:N

- Связь M:N означает, что одна строка таблицы A может быть связана со многими строками таблицы B, и каждая строка B также может быть связана со многими строками A.
- Один Order содержит несколько Products.
- Один Product может входить в множество Orders.
- Одного внешнего ключа в orders или products недостаточно.
- Для связи создаётся отдельная промежуточная таблица.
- Она хранит пары внешних ключей.

Почему FOREIGN KEY в одной таблице не решает M:N

product_id в orders

- Один заказ сможет ссылаться только на один товар
- Нельзя удобно хранить несколько товаров
- Не соответствует предметной области

order_id в products

- Один товар сможет относиться только к одному заказу
- Товар нельзя использовать в других заказах
- Не соответствует предметной области

Промежуточная таблица

- Для Orders ↔ Products создаём order_items.
- order_items содержит order_id.
- order_items содержит product_id.
- Каждая строка означает: этот товар входит в этот заказ.
- Промежуточная таблица может хранить дополнительные свойства самой связи.

Что хранит order_items

Столбец	Назначение
order_id	Какому заказу принадлежит позиция
product_id	Какой товар находится в заказе
quantity	Количество единиц товара
price	Цена товара в момент оформления заказа

Многие строки одной таблицы связаны со многими строками другой с помощью промежуточной таблицы.



orders

Заказы

id PK	user_id	status	created_at
1	3	new	2026-09-10
2	1	paid	2026-09-11
3	3	delivered	2026-09-12
4	2	new	2026-09-13
...	...	...	...

Таблица **orders** хранит заказы.
Каждый заказ может содержать несколько товаров.

1

N

Один заказ
может иметь
много позиций
(order_items).



order_items

Позиции заказа (промежуточная таблица)

order_id PK, FK	product_id PK, FK	quantity	price
1	1	1	15000
1	2	2	8000
1	3	1	85000
2	1	1	15000
3	2	1	8000
3	4	3	65000
...	...	...	...

Одна строка **order_items** означает,
что конкретный товар входит в конкретный заказ
(с указанием количества и цены).

N

1

Один товар
может быть
включён
во многие
заказы
(order_items).



products

Товары

id PK	name	price	category_id
1	Keyboard	15000	1
2	Mouse	8000	1
3	Monitor	85000	2
4	SSD 2TB	65000	3
...	...	...	...

Таблица **products** хранит товары.
Один и тот же товар может входить
во множество заказов.



Что происходит?

- Связь M:N между orders и products реализуется через промежуточную таблицу order_items.
- Между orders и order_items — связь 1:N.
- Между products и order_items — связь 1:N.
- Такая структура позволяет хранить дополнительные данные о позиции заказа (количество, цену и др.).



Одна таблица — одна пара

Каждая строка **order_items** связывает один заказ с одним товаром и содержит свойства этой связи.

Пример: в заказе №1 товар №2
в количестве 2 по цене 8000.



Почему не добавить product_id в orders?

- В заказе может быть несколько товаров.
- Один товар может входить во многие заказы.
- Один внешний ключ в одной из таблиц не решает связь многие ко многим.
- Нужна промежуточная таблица order_items.



M:N = две связи 1:N через промежуточную таблицу

Промежуточная таблица превращает связь многие ко многим в две простые связи один ко многим.



Больше возможностей
для вашего магазина!

Таблица order_items

SQL

```
CREATE TABLE order_items (  
  order_id INTEGER NOT NULL REFERENCES orders(id),  
  product_id INTEGER NOT NULL REFERENCES products(id),  
  quantity INTEGER NOT NULL CHECK (quantity > 0),  
  price INTEGER NOT NULL CHECK (price > 0),  
  PRIMARY KEY (order_id, product_id)  
);
```

Что важно

- order_id является внешним ключом на orders.
- product_id является внешним ключом на products.
- Составной PRIMARY KEY запрещает повтор одной пары order_id + product_id.
- quantity описывает свойства связи Order ↔ Product.
- price сохраняет цену товара в конкретном заказе.

Как работает связь M:N

- 1 Создаётся Order**

Например, новый заказ получает id = 10.
- 2 Выбираются Products**

Пользователь выбирает товары с id 2, 5 и 8.
- 3 Создаются строки order_items**

В таблицу добавляются пары (10,2), (10,5) и (10,8).
- 4 Связь становится двусторонней**

По order_id можно найти товары заказа, а по product_id — заказы, содержащие товар.



Учебный вопрос 5. Связи в Internet Shop

Объединяем все типы отношений в одной схеме

Связи нашей базы данных

Сущности	Тип	Реализация
User ↔ User Profile	1:1	user_profiles.user_id PK + FK
Category → Products	1:N	products.category_id
User → Orders	1:N	orders.user_id
Orders ↔ Products	M:N	order_items

Как определить тип связи

- Сформулируйте правило предметной области обычными словами.
- Спросите: сколько объектов В может относиться к одному А?
- Затем спросите: сколько объектов А может относиться к одному В?
- Ответы определяют 1:1, 1:N или M:N.
- Только после этого решайте, где хранить FOREIGN KEY.

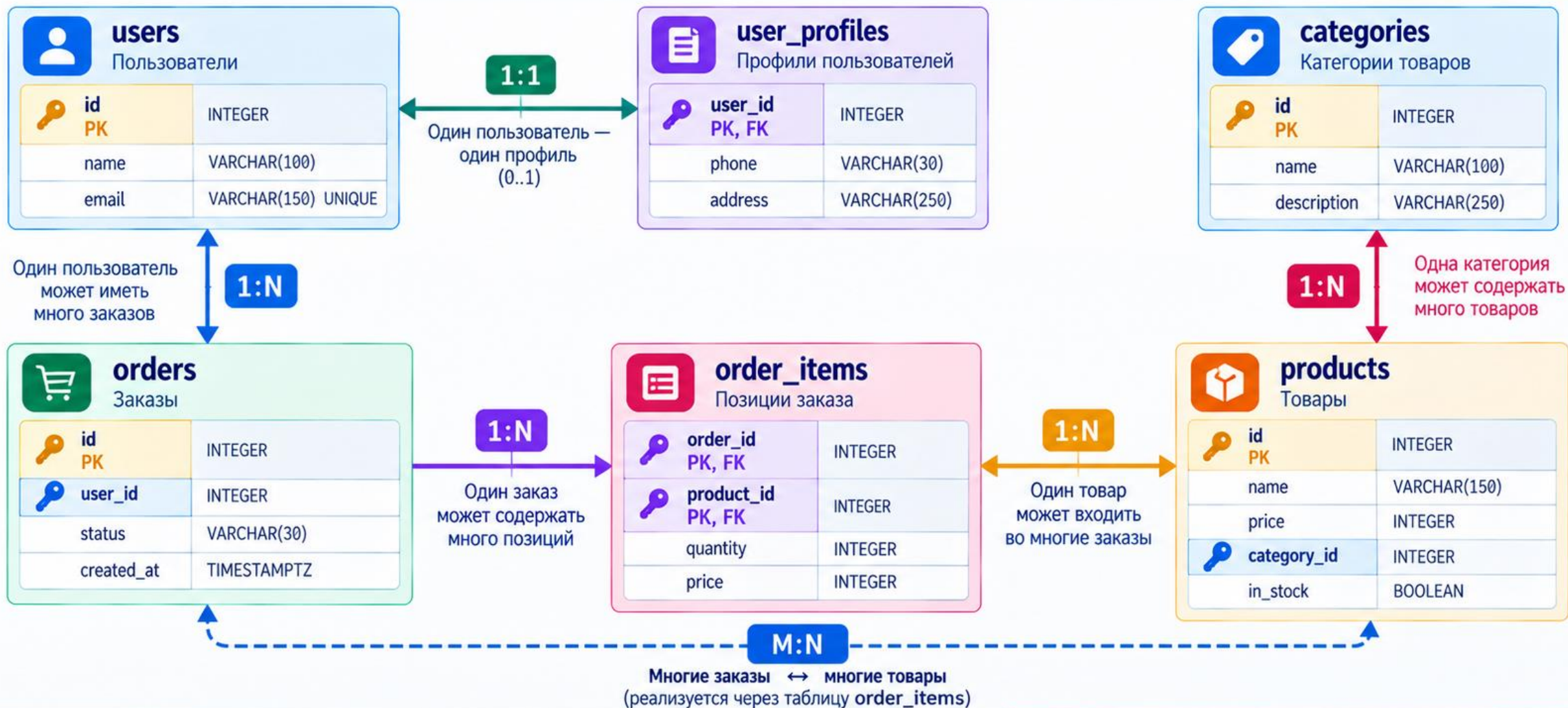
Алгоритм проектирования связи

- 1** **Выбрать две сущности** Например, Category и Product.
- 2** **Сформулировать бизнес-правило** Одна категория может содержать много товаров.
- 3** **Определить кардинальность** Category → Product имеет тип 1:N.
- 4** **Определить расположение FOREIGN KEY** Для 1:N внешний ключ размещается на стороне N: products.category_id.
- 5** **Добавить ограничение** FOREIGN KEY должен ссылаться на PRIMARY KEY родительской таблицы.
- 6** **Проверить тестовыми данными** База должна принимать корректные ссылки и отклонять некорректные.

Что происходит при удалении связанной строки

- FOREIGN KEY не позволяет нарушить ссылочную целостность.
- Удаление родительской строки может быть запрещено, если на неё существуют ссылки.
- Для некоторых связей можно явно настроить ON DELETE CASCADE.
- CASCADE автоматически удаляет зависимые строки.
- Поведение удаления должно определяться смыслом данных, а не удобством написания SQL.

Все основные таблицы и связи между ними. Заказы и товары связаны по схеме M:N через промежуточную таблицу order_items.



Обозначения:



PK — первичный ключ



FK — внешний ключ



— связь



— связь многие ко многим (через промежуточную таблицу)



Единая схема — целостные данные

Внешние ключи обеспечивают ссылочную целостность, а правильные связи отражают бизнес-правила интернет-магазина.

Получаем товары конкретного заказа

SQL

```
SELECT
    p.id,
    p.name,
    oi.quantity,
    oi.price
FROM order_items AS oi
JOIN products AS p
    ON p.id = oi.product_id
WHERE oi.order_id = $1
ORDER BY p.name;
```

Что важно

- order_items определяет товары конкретного заказа.
- JOIN получает данные Product.
- \$1 содержит id заказа.
- quantity и price принадлежат позиции заказа.
- Один запрос восстанавливает данные из связанных таблиц.

Получаем заказы, содержащие товар

SQL

```
SELECT
    o.id,
    o.status,
    o.created_at,
    oi.quantity
FROM order_items AS oi
JOIN orders AS o
    ON o.id = oi.order_id
WHERE oi.product_id = $1
ORDER BY o.created_at DESC;
```

Что важно

- Ту же промежуточную таблицу можно использовать в обратном направлении.
- \$1 содержит id товара.
- Запрос находит все заказы, содержащие этот товар.
- Это демонстрирует двустороннюю природу связи M:N.

Определяем тип связи по правилам предметной области, а затем реализуем её с помощью PRIMARY KEY и FOREIGN KEY.



Начните с бизнес-правила и задайте два вопроса о кардинальности.



Ответы на два вопроса однозначно определяют тип связи.

1. Сколько B может относиться к одному A?

Только один
(не более одного)

Много
(несколько)

2. Сколько A может относиться к одному B?

Только один
(не более одного)

Много
(несколько)

2. Сколько A может относиться к одному B?

Только один
(не более одного)

Много
(несколько)

1:1

Один к одному

Одной строке таблицы A соответствует максимум одна строка таблицы B, и наоборот.

Пример: User ↔ User Profile

users	user_profiles
 id PK	 user_id PK, FK
name	phone
email	address

1:1

1:N

Один ко многим

Одной строке таблицы A может соответствовать много строк таблицы B. Каждая строка B относится только к одной строке A.

Пример: Category → Products

categories	products
 id PK	 id PK
name	name
description	 category_id FK
	price

1:N

N:1

Многие к одному

Много строк таблицы A относятся к одной строке таблицы B. (Это тот же тип 1:N, но с обратной стороны.)

Пример: Orders → User

orders	users
 id PK	 id PK
 user_id FK	name
status	email
created_at	

N:1

M:N

Многие ко многим

Много строк таблицы A могут быть связаны со многими строками таблицы B. Требуется промежуточная таблица.

Пример: Orders ↔ Products

orders	order_items	products
 id PK	 order_id FK	 id PK
	 product_id FK	
	quantity	
	price	

1:N 1:N



Сначала бизнес-правило — потом FOREIGN KEY

Определите кардинальность по смыслу предметной области, затем спроектируйте таблицы и добавьте ограничения PRIMARY KEY и FOREIGN KEY.



Правильные связи — основа целостной базы данных.

Мини-практика: добавьте связи Internet Shop

Расширьте shop_db таблицами users, user_profiles, orders и order_items и реализуйте связи 1:1, 1:N и M:N.

Задание

- Создать users
- Создать user_profiles с user_id как PK и FK
- Создать orders с user_id
- Создать order_items с order_id и product_id
- Добавить тестовых пользователей и заказы
- Добавить товары в заказы через order_items
- Проверить ограничения внешних ключей
- Выполнить JOIN для получения состава заказа

Критерии проверки

- User ↔ User Profile реализовано как 1:1
- User → Orders реализовано как 1:N
- Category → Products продолжает работать как 1:N
- Orders ↔ Products реализовано через order_items
- Несуществующие внешние ключи отклоняются PostgreSQL
- JOIN возвращает товары выбранного заказа

Контрольные вопросы

1. Что такое кардинальность связи?
2. Чем 1:1 отличается от 1:N?
3. На какой стороне связи 1:N располагается FOREIGN KEY?
4. Как обеспечить не более одного профиля для пользователя?
5. Почему M:N нельзя реализовать одним обычным FOREIGN KEY?
6. Для чего нужна промежуточная таблица?
7. Какие внешние ключи содержит order_items?
8. Зачем в order_items хранить quantity?
9. Что произойдёт при попытке записать несуществующий внешний ключ?
10. Как определить тип связи по правилам предметной области?

Итоги занятия

1

Кардинальность определяет допустимое количество связанных строк.
Связь 1:1 можно реализовать

2

уникальным FOREIGN KEY или ключом, одновременно являющимся PRIMARY KEY.

3

В связи 1:N внешний ключ хранится на стороне N.

4

Category → Products и User → Orders являются примерами 1:N.

5

Связь M:N требует промежуточной таблицы.

6

Orders ↔ Products реализуется через order_items.

7

Промежуточная таблица может хранить свойства самой связи, например quantity и price.

8

FOREIGN KEY обеспечивает ссылочную целостность.

9

Тип связи определяется правилами предметной области, а не удобством написания кода.